

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине «Объектно-ориентированное программирование»
Тема: Создание игры в стиле ООП

Студентка гр. 4342

_____ Киселева А.Д.

Преподаватель

_____ Жангиров Т.Р.

Санкт-Петербург

2025

ВВЕДЕНИЕ

Цель: разработка модульной и расширяемой архитектуры пошаговой тактической игры с применением современных принципов объектно-ориентированного программирования на C++, обеспечивающей гибкость модификации игровой логики и эффективное управление ресурсами.

Задачи:

1. Реализация класса *Player*;
2. Реализация класса *Enemy*;
3. Создание класса игрового поля *Field*;
4. Реализация класса клетки поля *Cell*;
5. Реализация семантики перемещения и копирования;
6. Реализация системы препятствий;
7. Реализация системы боя;
8. Реализация замедляющих клеток;

ОПИСАНИЕ СТРУКТУРЫ КОДА

Для удобного перемещения по игровому полю и хранения информации о состоянии его составляющих реализован класс *Cell*.

Поля класса:

- *typeOfCell type* — тип клетки.
- *character person* — тип персонажа, находящегося на клетке в данный момент.

Методы класса:

1. *Cell()* - конструктор по умолчанию. Инициализирует клетку в "пустом" состоянии. Устанавливает тип клетки как *DEFAULT* (проходимая клетка) и указывает, что на клетке отсутствуют персонажи (*NOBODY*).
2. *Cell(typeOfCell type)* - параметризованный конструктор. Создает клетку с заданным типом, но без персонажа. Используется при инициализации поля с различными типами клеток (препятствия, замедляющие клетки).
3. *void setType(typeOfCell newType)* — метод изменения типа клетки.

Принимаемый аргумент:

typeOfCell newType — новый тип клетки.

4. *void setCharacter(character newPerson)* — метод изменения персонажа, находящегося на клетке.

Принимаемый аргумент:

character newPerson — новый тип персонажа.

5. *typeOfCell getType() const* — метод получения типа клетки.

Возвращаемое значение:

typeOfCell type — тип клетки.

6. *character getCharacter() const* — метод получения типа персонажа.

Возвращаемое значение:

character person — тип персонажа.

Класс *Player* реализует логику управления основными характеристиками игрока — его здоровьем, счетом и типом атаки, а также положением на поле и определением возможности передвижения.

Поля класса:

- *int health* — текущее количество здоровья игрока.
- *int score* — счет игрока, увеличивающийся при победе над врагами.
- *bool moveAbility* — возможность передвижения игрока.
- *typeOfFight damage* — тип текущей атаки игрока, влияющий на наносимый урон.
- *std::pair<int, int> coordinates* — координаты игрока на поле.

Методы класса:

1. *Player()* - конструктор по умолчанию. Инициализирует нового игрока со стандартными значениями: *health = 10*, *score = 0*, *damage = CLOSE*, *coordinates = {0, 0}*, *moveAbility = 1*. Обеспечивает корректное начальное состояние объекта.

2. *int getHealth() const* - метод получения текущего здоровья.

Возвращаемое значение:

int health — текущее здоровье игрока.

3. *void setHealth(int newHealth)* - метод установки нового значения здоровья.

Принимаемый аргумент:

int newHealth — новое значение здоровья.

4. *int getScore() const* - метод получения текущего счета.

Возвращаемое значение:

int score — текущий счет.

5. *void setScore(int newScore)* - метод установки нового значения счета.

Принимаемый аргумент:

int newScore — новое значение счета.

6. *typeOfFight getDamage() const* - метод получения типа текущей атаки.

Возвращаемое значение:

typeOfFight damage — тип текущей атаки.

7. *void setDamage(typeOfFight newDamage)* — метод установки типа атаки.

Принимаемый аргумент:

typeOfFight newDamage — новый тип атаки.

8. *std::pair<int, int> getCoordinates() const* — метод получения текущих координат на поле.

Возвращаемое значение:

std::pair<int, int> current_coords — координаты клетки поля.

9. *void setCoordinates(int x, int y)* — метод установки новых координат положения игрока.

Принимаемые аргументы:

int x, int y — координаты новой клетки.

10. *bool getMoveAbility() const* — метод для получения информации о возможности перемещения по полю. Используется для пропуска хода при нахождении на замедляющей клетке.

Возвращаемое значение:

True — если игрок может передвигаться, иначе *False*.

11. *void setMoveAbility(bool value)* — метод для изменения возможности перемещения по полю.

Принимаемый аргумент:

bool value — новое значение (*True/False*)

12. *bool isAlive()* - метод, проверяющий состояние здоровья игрока.

Возвращаемое значение:

True — если здоровье положительное, иначе *False*.

13. *bool selectCombatMode(typeOfFight mode)* — метод для изменения типа атаки игрока.

Принимаемый аргумент:

typeOfFight mode — тип атаки, на которую будет заменена текущая.

Возвращаемое значение:

True — если выбран тип атаки, отличающийся от текущего, иначе *False*.

Класс *Enemy* реализует логику управления основными характеристиками врага — его здоровьем, наносимым уроном, положением на игровом поле.

Поля класса:

- *int health* — текущее количество здоровья врага.
- *int damage* — количество урона, которое враг наносит игроку при атаке.
- *std::pair<int, int> coordinates* — текущие координаты клетки врага.

Методы класса:

1. *Enemy()* - конструктор по умолчанию. Инициализирует нового врага со стандартными значениями: *health = 5, damage = 1, coordinates = {-1, -1}*.
2. *int getHealth() const* - метод получения текущего здоровья врага.

Возвращаемое значение:

int health — текущее здоровье врага.

3. *void setHealth(int newHealth)* - метод установки нового значения здоровья врага.

Принимаемый аргумент:

int newHealth — новое значение здоровья.

4. *int getDamage() const* - метод получения значения наносимого урона.

Возвращаемое значение:

int damage — количество урона, которое враг наносит за одну атаку.

5. *std::pair<int, int> getCoordinates() const* — метод получения текущих координат врага.

Возвращаемое значение:

std::pair<int, int> current_coords — текущие координаты.

6. *void setCoordinates(int x, int y)* — метод установки новых координат врага.

Принимаемые аргументы:

int x, int y — координаты новой клетки.

7. *bool isAlive()* - метод, проверяющий состояние здоровья врага.

Возвращаемое значение:

True — если здоровье положительное, иначе *False*.

Класс *Field* реализует логику игрового поля, включая его создание, обработку перемещений игрока и врага.

Поля класса:

- *std::vector<std::vector<Cell*>> area* - двумерный массив указателей на клетки поля, представляющий игровое пространство.
- *int length* - длина игрового поля в клетках.
- *int width* - ширина игрового поля в клетках.

Методы класса:

1. *Field(int length, int width)* - конструктор с параметрами. Создает игровое поле заданного размера (от 10x10 до 25x25 клеток). Инициализирует поле клетками различных типов (OBSTACLE, FREEZE, DEFAULT) согласно вероятностному распределению, затем размещает игрока на левой верхней клетке.
2. *Field(const Field& field)* - конструктор копирования. Создает глубокую копию игрового поля, включая копирование всех клеток и объектов.

3. *Field(Field&& field)* - конструктор перемещения. Эффективно переносит ресурсы из временного объекта, обнуляя исходный объект.
4. *Field& operator = (const Field& field)* - оператор присваивания с копированием.
5. *Field& operator = (Field&& field)* - оператор присваивания с перемещением.
6. *character getCellCharacter(int x, int y) const* — метод для определения типа персонажа на конкретной клетке поля.

Принимаемые аргументы:

int x, int y — координаты клетки.

Возвращаемое значение:

character cell_character — персонаж выбранной клетки.

7. *void setCellCharacter(int x, int y, character newCharacter)* — метод для смены персонажа на конкретной клетке.

Принимаемые аргументы:

int x, int y — координаты выбранной клетки.

character newCharacter — новый персонаж.

8. *typeOfCell getCellType(int x, int y) const* — метод определения типа конкретной клетки поля.

Принимаемые аргументы:

int x, int y — координаты выбранной клетки.

9. *int getLength() const* - метод получения длины поля.

Возвращаемое значение:

int length - длина игрового поля.

10. *int getWidth() const* - метод получения ширины поля.

Возвращаемое значение:

int width - ширина игрового поля.

11. *bool canMoveTo(int x, int y)* - метод проверки возможности перемещения в указанную клетку.

Принимаемые аргументы:

int x, int y — координаты целевой клетки

Возвращаемое значение:

bool - *true* если перемещение возможно, *false* если клетка недоступна

12. *~Field()* - деструктор. Обеспечивает корректное освобождение памяти, занятой клетками поля.

Класс *GameManager* реализует связь между классами игрока, врага и поля, управляет ходами, атаками и перемещениями героев.

Поля класса:

- *Player player* — объект игрока;
- *Enemy enemy* — объект врага;
- *Field field* — игровое поле;
- *int moves* — счетчик ходов.

Методы класса:

1. *GameManager()* - конструктор по умолчанию. Инициализирует игрока, врага, поле размером 15×15 и счетчик ходов.
2. *Player getPlayer() const* — метод получения объекта игрока.

Возвращаемое значение:

Player player - текущий игрок.

3. *void setPlayer(Player newPlayer)* — метод установки нового игрока. Нужен для обновления характеристик первоначального игрока в ходе игры.

Принимаемый аргумент:

Player newPlayer - новый объект игрока.

4. *Enemy getEnemy() const* — метод получения объекта врага.

Возвращаемое значение:

Enemy enemy - текущий враг.

5. *Field getField() const* — метод получения игрового поля.

Возвращаемое значение:

Field field - текущее игровое поле.

6. *int getMoves() const* — метод получения количества ходов.

Возвращаемое значение:

int moves - текущее количество ходов.

7. *void setMoves(int newMoves)* — метод установки количества ходов.

Принимаемый аргумент:

int newMoves - новое значение счетчика ходов.

8. *void placeEnemy()* - метод случайного размещения врага на свободной клетке поля.

9. *bool isEnemyInRange(int range)* — метод проверки нахождения врага на определенном расстоянии для атаки.

Принимаемый аргумент:

int range — расстояние, нужное для определенного типа атаки.

Возвращаемое значение:

True - если враг находится на указанном расстоянии от игрока, иначе

False.

10. *void attackEnemy()* - атака врага игроком. Проверяет возможность атаки (дистанцию и тип урона), наносит урон врагу. При убийстве врага увеличивает счет игрока и размещает нового врага.

11. *void movePlayer(char direction)* — метод перемещения игрока. Проверяет возможность перемещения на выбранную клетку, обрабатывает эффекты клеток (заморозка) и обновляет счетчик ходов.

Принимаемый аргумент:

char direction - направление движения ('w', 's', 'a', 'd').

12. *void attackPlayer()* - атака игрока врагом, наносит урон игроку.

13. *void moveEnemy()* - перемещение врага. Реализует поиск кратчайшего пути к игроку и атакует его при возможности.

ОБОСНОВАНИЕ АРХИТЕКТУРНЫХ РЕШЕНИЙ

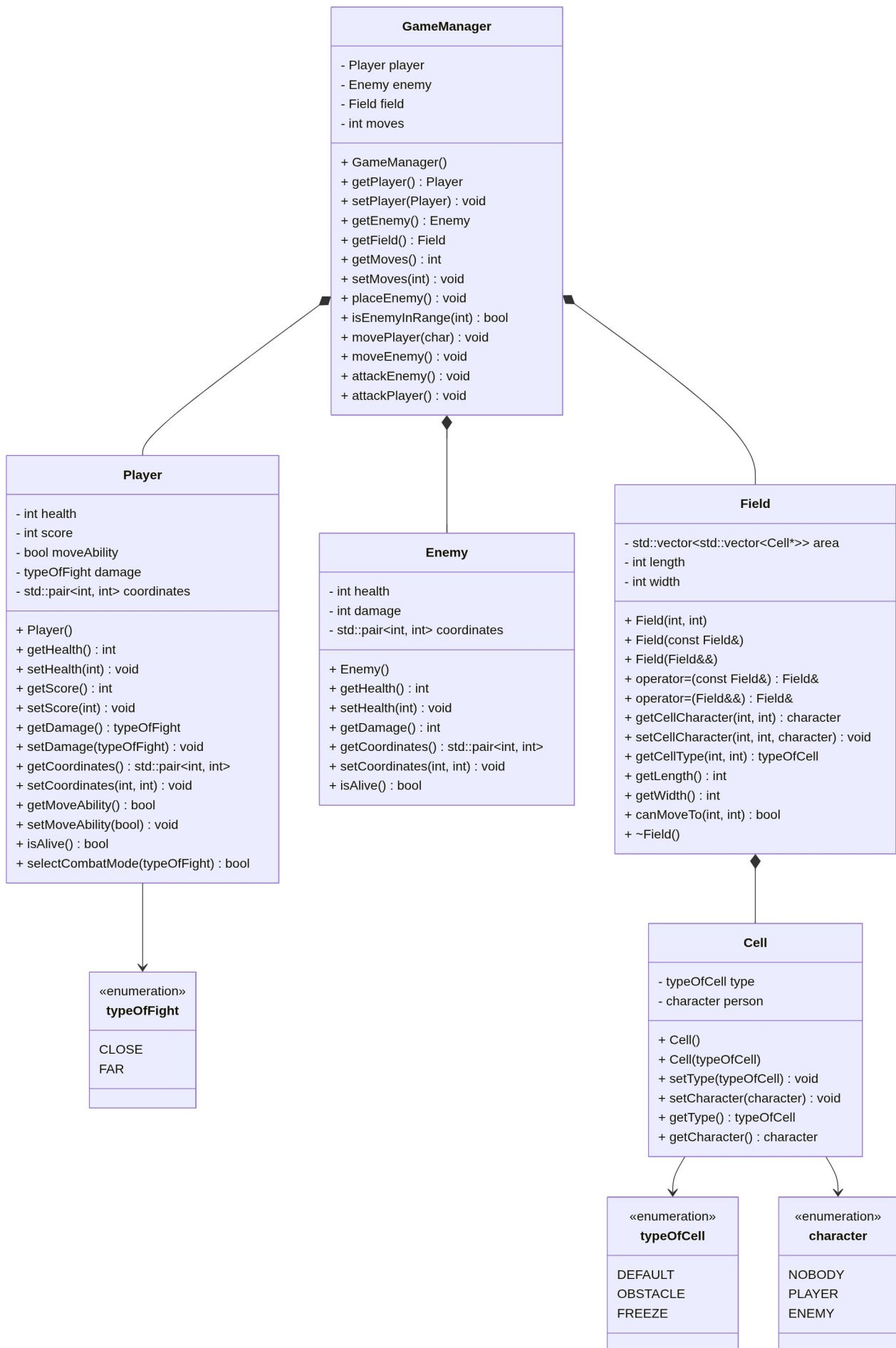
Класс *Cell* является основным строительным блоком, так как представляет элементарную единицу игрового пространства. Объект имеет двойную систему состояния, то есть хранит как тип самой клетки так и информацию о находящемся персонаже. Класс предназначен для массового создания клеток, поэтому содержит в себе только простые методы (геттеры и сеттеры для приватных полей) для их последующего использования при создании игрового поля.

Классы *Player* и *Enemy* являются контейнерами данных и отвечают за хранение и модификацию характеристик игровых сущностей. Сущности хранят только состояния (здоровье, урон, очки, координаты, возможность передвижения).

Класс *Field* реализует логику игрового поля - двумерной сетки, состоящей из клеток. С помощью рандомайзера при создании поля генерируются препятствия и замедляющие клетки, а также определяется начальная позиция игрока. При перемещении персонажей проводится проверка возможности перемещения на определенные клетки, обновляются координаты персонажей и состояние соответствующих клеток.

Класс *GameManager* содержит большую часть игровой логики, координирует взаимодействие между сущностями (атака, перемещение, изменение типов клеток поля). В классе создаются объекты сущностей, которые впоследствии обрабатываются как текущем классе, так и в других классах.

Взаимосвязь классов представлена на UML-диаграмме:



Жизненный цикл клеток зависит от поля. При создании экземпляра поля создается множество экземпляров клеток, при уничтожении поля уничтожаются все его клетки. Поэтому *Field* и *Cell* имеют композиционную связь.

GameManager содержит по одному экземпляру игрока, врага и поля, причем жизненный цикл всех этих объектов зависит от жизни *GameManager*. Значит *GameManager* и *Player/Enemy/Field* имеют композиционную зависимость.

Cell использует перечисления *typeOfCell* и *character* как типы своих полей, следовательно тип связи между этими перечислениями и *Cell* — зависимость. Аналогично зависимость между *Player* и *typeOfFight*.

ТЕСТИРОВАНИЕ

```
#include <gtest/gtest.h>
#include "../include/cell.hpp"
#include "../include/player.hpp"
#include "../include/enemy.hpp"
#include "../include/field.hpp"
#include "../include/gameManager.hpp"

TEST(CellTest, ConstructorWithType) {
    Cell cell(typeOfCell::OBSTACLE);
    EXPECT_EQ(cell.getType(), typeOfCell::OBSTACLE);
    EXPECT_EQ(cell.getCharacter(), character::NOBODY);
}

TEST(CellTest, SetCharacter) {
    Cell cell;
    cell.setCharacter(character::PLAYER);
    EXPECT_EQ(cell.getCharacter(), character::PLAYER);
}

TEST(CellTest, SetType) {
    Cell cell;
    cell.setType(typeOfCell::FREEZE);
    EXPECT_EQ(cell.getType(), typeOfCell::FREEZE);
}

TEST(PlayerTest, IsAlive) {
    Player player;
    EXPECT_TRUE(player.isAlive());
}
```

```

    player.setHealth(0);
    EXPECT_FALSE(player.isAlive());

    player.setHealth(-5);
    EXPECT_FALSE(player.isAlive());
}

TEST(PlayerTest, SelectCombatMode) {
    Player player;

    EXPECT_EQ(player.getDamage(), typeOfFight::CLOSE);

    bool changed = player.selectCombatMode(typeOfFight::FAR);
    EXPECT_TRUE(changed);
    EXPECT_EQ(player.getDamage(), typeOfFight::FAR);

    changed = player.selectCombatMode(typeOfFight::FAR);
    EXPECT_FALSE(changed);
    EXPECT_EQ(player.getDamage(), typeOfFight::FAR);
}

TEST(PlayerTest, MoveAbility) {
    Player player;
    EXPECT_TRUE(player.getMoveAbility());

    player.setMoveAbility(false);
    EXPECT_FALSE(player.getMoveAbility());
}

TEST(EnemyTest, IsAlive) {
    Enemy enemy;
    EXPECT_TRUE(enemy.isAlive());
}

```



```

        enemy.setHealth(0);
        EXPECT_FALSE(enemy.isAlive());

        enemy.setHealth(-3);
        EXPECT_FALSE(enemy.isAlive());
    }

TEST(EnemyTest, CoordinatesManagement) {
    Enemy enemy;
    auto coords = enemy.getCoordinates();
    EXPECT_EQ(coords.first, -1);
    EXPECT_EQ(coords.second, -1);

    enemy.setCoordinates(5, 10);
    coords = enemy.getCoordinates();
    EXPECT_EQ(coords.first, 5);
    EXPECT_EQ(coords.second, 10);
}

TEST(FieldTest, Constructor) {
    Field validField(15, 15);
    EXPECT_EQ(validField.getLength(), 15);
    EXPECT_EQ(validField.getWidth(), 15);
}

TEST(FieldTest, CanMoveTo) {
    Field field(15, 15);

    EXPECT_TRUE(field.canMoveTo(0, 0));
    EXPECT_TRUE(field.canMoveTo(14, 14));
}

```

```

    EXPECT_FALSE(field.canMoveTo(-1, 0));
    EXPECT_FALSE(field.canMoveTo(0, 15));
}

TEST(FieldTest, CellCharacterManagement) {
    Field field(15, 15);

    EXPECT_EQ(field.getCellCharacter(0, 0), character::PLAYER);

    field.setCellCharacter(5, 5, character::ENEMY);
    EXPECT_EQ(field.getCellCharacter(5, 5), character::ENEMY);

    EXPECT_EQ(field.getCellCharacter(-1, -1), character::NOBODY);
    EXPECT_EQ(field.getCellCharacter(20, 20), character::NOBODY);
}

TEST(FieldTest, CellTypeManagement) {
    Field field(15, 15);

    typeOfCell type = field.getCellType(0, 0);
    EXPECT_TRUE(type == typeOfCell::DEFAULT || type ==
typeOfCell::FREEZE);

    EXPECT_EQ(field.getCellType(-1, -1), typeOfCell::OBSTACLE);
}

TEST(FieldTest, CopyConstructor) {
    Field original(15, 15);
    original.setCellCharacter(7, 7, character::ENEMY);

    Field copy(original);

```

```

    EXPECT_EQ(copy.getLength(), original.getLength());
    EXPECT_EQ(copy.getWidth(), original.getWidth());
    EXPECT_EQ(copy.getCellCharacter(7, 7), character::ENEMY);
}

TEST(GameManagerTest, InitialState) {
    GameManager game;

    EXPECT_EQ(game.getMoves(), 0);
    EXPECT_TRUE(game.getPlayer().isAlive());
    EXPECT_TRUE(game.getEnemy().isAlive());
}

TEST(GameManagerTest, PlaceEnemy) {
    GameManager game;
    game.placeEnemy();

    auto enemyCoords = game.getEnemy().getCoordinates();
    EXPECT_EQ(game.getField().getCellCharacter(enemyCoords.first,
enemyCoords.second), character::ENEMY);
}

TEST(GameManagerTest, EnemyInRange) {
    GameManager game;

    game.getPlayer().setCoordinates(5, 5);
    game.getEnemy().setCoordinates(6, 6);

    EXPECT_TRUE(game.isEnemyInRange(2)); // Расстояние = 2
    EXPECT_FALSE(game.isEnemyInRange(1)); // Расстояние = 2 > 1
}

```

```

TEST(GameManagerTest, PlayerMovement) {
    GameManager game;
    int initialMoves = game.getMoves();

    game.movePlayer('d');

    EXPECT_GT(game.getMoves(), initialMoves);
    auto playerCoords = game.getPlayer().getCoordinates();
    EXPECT_EQ(playerCoords, std::make_pair(0, 1));
}

```

```

TEST(GameManagerTest, PlayerAttackEnemy) {
    GameManager game;

    game.getPlayer().setCoordinates(5, 5);
    game.getEnemy().setCoordinates(6, 5);
    game.getEnemy().setHealth(2);
    game.getPlayer().setDamage(typeOfFight::CLOSE);

    int initialMoves = game.getMoves();
    game.attackEnemy();

    EXPECT_EQ(game.getEnemy().getHealth(), 5);
}

```

```

TEST(GameManagerTest, EnemyRespawn) {
    GameManager game;

    game.getEnemy().setHealth(1);
    game.attackEnemy();

    EXPECT_GT(game.getEnemy().getHealth(), 0);
}

```

```
}
```

```
TEST(GameManagerTest, BoundaryCases) {  
    GameManager game;  
  
    game.getPlayer().setCoordinates(0, 0);  
    game.movePlayer('a');  
    EXPECT_EQ(game.getPlayer().getCoordinates(), std::make_pair(0,  
0));  
}
```

```
int main(int argc, char **argv) {  
    ::testing::InitGoogleTest(&argc, argv);  
    return RUN_ALL_TESTS();  
}
```

Запуск тестов:

```
[=====] 20 tests from 5 test suites ran. (1 ms total)  
[  PASSED  ] 20 tests.
```

Все проверки пройдены, программа работает корректно.

ЗАКЛЮЧЕНИЕ

В результате лабораторной работы была создана система взаимосвязанных классов, каждый из которых выполняет строго определенную функцию и может быть модифицирован независимо от других компонентов системы; реализованы конструкторы и операторы копирования/перемещения, обеспечивающие безопасное глубокое копирование элементов игрового поля; соблюдены принципы инкапсуляции, исключено дублирование данных, предотвращены небезопасные операции с указателями. Программа протестирована и работает корректно.